

Instituto Tecnológico de Colima

Ingeniería en Inteligencia Artificial

Cómputo en la Nube

Proyecto Nebula Arena — Unidad 3

Práctica 3.6 — Infraestructura como código

Jarom Cortes Perez

Carlos Antonio Vogel Romero

11 de noviembre de 2026

Introducción

En las prácticas anteriores la nube tenía muchas piezas: red, clúster, nodos, Agones, monitoreo, registros de imágenes, permisos y la tabla de líderes. Crear todo eso a mano en la consola es lento, no se puede repetir igual y es fácil olvidar algo encendido que siga cobrando. Con Terraform describimos toda la infraestructura en archivos de código: un comando la crea, otro la borra, y los cambios se revisan antes de aplicarlos.

Objetivo

Crear con Terraform todo el entorno de Nebula Arena, revisar el plan antes de aplicarlo, comprobar que el clúster funciona y destruirlo al terminar sin dejar recursos que sigan cobrando.

Lo que hicimos

1. Escribir la infraestructura

Usamos la variante de AWS del material, en la carpeta `infra-aws/`. Separamos la infraestructura por archivos:

Archivo	Qué crea
`red.tf`	La red, dos subredes públicas y los puertos 7000–8000 para las partidas
`cluster.tf`	El clúster EKS, los nodos (2 a 6) y los permisos del Cluster Autoscaler
`complementos.tf`	Agones, el Cluster Autoscaler, Prometheus y Grafana con Helm
`registro.tf`	Los repositorios de imágenes en ECR
`tabla_lideres.tf`	La tabla de líderes de la práctica 3.5
`github.tf`	El usuario del pipeline de la práctica 3.2
`presupuesto.tf`	Una alarma de costos

En total son **46 recursos**. Todo lleva la etiqueta `proyecto=nebula-arena`, para poder encontrarlo después.

2. Estado remoto

Terraform guarda en un archivo de estado qué recursos creó. Lo movimos a un bucket de S3 privado, versionado y cifrado, con bloqueo para que dos personas no apliquen cambios al mismo tiempo. Usamos el bloqueo nativo de S3 (`use_lockfile`, desde Terraform 1.10), así que no hizo falta la tabla de DynamoDB del material.

3. Alarma de costos

Con el plan gratuito todo se descuenta de los créditos y el costo siempre aparece en cero, así que una alarma normal nunca avisaría. Creamos un presupuesto que mide el costo **antes** de créditos,

con un límite de USD 15 al mes y avisos por correo al 50 % y al 80 % del gasto, y al 100 % del pronóstico.

4. Revisar el plan antes de aplicar

Siempre trabajamos con `terraform plan -out` y después aplicamos ese archivo, para que se aplique exactamente lo que revisamos. Después de la prueba de carga de la 3.4 corrimos el plan contra la infraestructura viva para buscar *drift* (cambios hechos a mano fuera de Terraform). No encontró ninguno; lo único nuevo era el presupuesto que acabábamos de agregar:

```
terraform plan -out completo.tfplan  (2026-11-09 14:52)
Estado remoto: s3://nebula-tfstate-********1139/nebula/aws.tfstate (bloqueo con use_lockfile)
[…]
  # aws_budgets_budget.nebula[0] will be created
  + resource "aws_budgets_budget" "nebula" {
      + account_id        = (known after apply)
[…]
Plan: 1 to add, 0 to change, 0 to destroy.
```

Figura 1. *terraform plan contra la infraestructura viva, con el estado remoto en S3: sin drift; lo único nuevo es el presupuesto.*

Lo aplicamos y corrimos un segundo plan para comprobar que Terraform es idempotente, es decir, que si no cambia nada en el código no propone cambios:

```
No changes. Your infrastructure matches the configuration.
and found no differences, so no changes are needed.
```

Figura 2. *Segundo terraform plan: No changes.*

Los cambios los revisamos en el *pull request* #2 de GitHub, de la rama `practica-3.6` hacia `main`. En la descripción pegamos la salida del plan y explicamos qué significa antes de aplicarla:

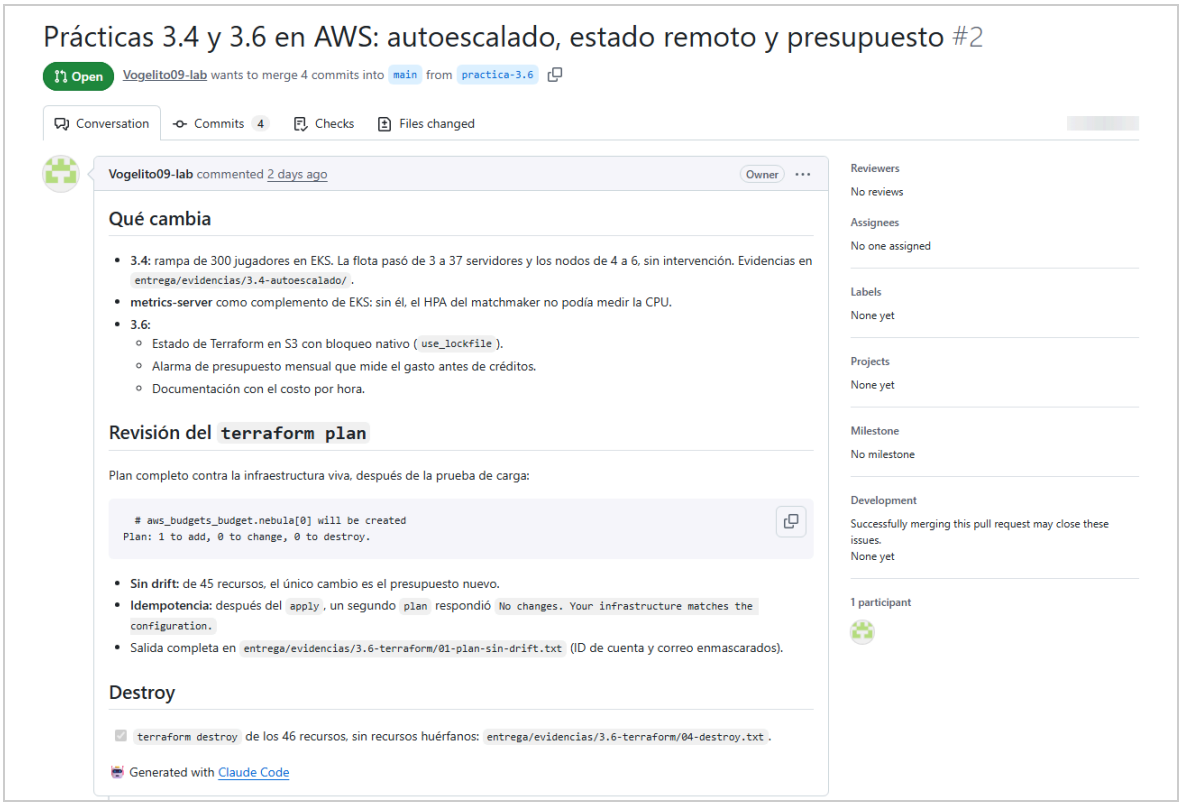


Figura 3. *Pull request #2 en GitHub con la revisión del terraform plan: sin drift (el único cambio es el presupuesto), idempotencia y destroy. Captura del 11 de noviembre.*

5. El clúster funcionando

Revisamos que lo que creó Terraform estuviera funcionando: el clúster activo, los nodos listos, Agones, el autoescalador, el monitoreo, la flota con sus 3 servidores libres y el matchmaker con su balanceador:

```
== Clúster creado por Terraform funcionando (2026-11-09 14:54) ==
nebula  1.36    ACTIVE  eks.17

# Nodos
NAME                                          STATUS   ROLES    AGE   VERSION               INSTANCE-TYPE
ip-10-0-12-209.us-east-2.compute.internal   Ready    <none>   11m   v1.36.5-eks-6e7947a   m7i-flex.large
ip-10-0-13-66.us-east-2.compute.internal    Ready    <none>   32m   v1.36.5-eks-6e7947a   m7i-flex.large
ip-10-0-2-183.us-east-2.compute.internal    Ready    <none>   36m   v1.36.5-eks-6e7947a   m7i-flex.large
ip-10-0-20-73.us-east-2.compute.internal    Ready    <none>   36m   v1.36.5-eks-6e7947a   m7i-flex.large
ip-10-0-31-58.us-east-2.compute.internal    Ready    <none>   32m   v1.36.5-eks-6e7947a   m7i-flex.large
[…]
NAME                            SCHEDULING   DESIRED   CURRENT   ALLOCATED   READY   AGE
fleet.agones.dev/nebula-fleet   Packed       3         3         0           3       31m

NAME                                                    AGE
fleetautoscaler.autoscaling.agones.dev/nebula-autoscaler   31m

NAME                                                REFERENCE               TARGETS       MINPODS   MAXPODS
REPLICAS   AGE
horizontalpodautoscaler.autoscaling/matchmaker-hpa   Deployment/matchmaker   cpu: 1%/60%   2         30
2          31m
NAME         TYPE           CLUSTER-IP      EXTERNAL-IP
PORT(S)        AGE
matchmaker   LoadBalancer   172.20.222.11   a6cd6956a66f14954a5b0517b09e7786-2138136351.us-east-2.elb.amazonaw
s.com   80:30080/TCP   31m
```

Figura 4. *Clúster creado por Terraform funcionando: EKS ACTIVE, 5 nodos Ready, la flota con 3 servidores Ready, el HPA y el balanceador del matchmaker.*

6. Destruir todo sin dejar huérfanos

Primero borramos el balanceador del matchmaker y la flota, porque esos los crea Kubernetes y no Terraform. Si no, el balanceador se queda huérfano y bloquea el borrado de la red. Después corrimos `terraform plan -destroy` (46 recursos) y lo aplicamos. El `destroy` nos encontró dos fallas, que corregimos en el código:

- **Orden de borrado:** Terraform borró la salida a internet antes que Agones, y el desinstalador de Agones ya no pudo descargar su imagen. Ahora los nodos dependen de esa ruta, así que la ruta se borra al final.
- **Usuario de GitHub:** la política de la cuenta gratuita no deja que Terraform borre el usuario con `force_destroy`. Le quitamos esa opción y borramos su llave con la línea de comandos antes del `destroy`.

Al final se destruyeron **46 de 46 recursos** y revisamos la cuenta servicio por servicio:

```
== terraform destroy (2026-11-09) ==

1. Antes del destroy se borraron el Service LoadBalancer del matchmaker y la flota
   (los crea Kubernetes, no Terraform; si no, el balanceador quedaría huérfano).
2. terraform plan -destroy -out destroy.tfplan  ->  Plan: 0 to add, 0 to change, 46 to destroy.
3. terraform apply destroy.tfplan  ->  45 recursos destruidos en 23 min. Dos problemas encontrados:
   a) El desinstalador de Agones (hook de Helm) no podía descargar su imagen porque
      Terraform ya había borrado la ruta a internet. Se borró el job atorado y se
      corrigió el código: el grupo de nodos ahora depende de la ruta (cluster.tf).
   b) force_destroy del usuario de GitHub intenta iam:DeleteLoginProfile, que la
      política de la organización (plan gratuito) niega. Se quitó force_destroy y
      su llave se borró con la CLI; un segundo plan/apply destruyó el usuario.
4. Total: 46 de 46 recursos destruidos. terraform state list -> vacío.

== Comprobación de recursos huérfanos (2026-11-09 15:25, us-east-2) ==

Clústeres EKS ..................... ninguno
Balanceadores (clásicos y v2) ..... ninguno
IPs elásticas ..................... ninguna
VPC no predeterminadas ............ ninguna
Repositorios ECR .................. ninguno
Lambda / DynamoDB / API Gateway ... ninguno
Usuarios y roles IAM 'nebula*' .... ninguno
Instancias EC2 activas ............ i-078b91404bcbe9360 (t3.micro "tablero-salome", creada el 2026-10-26,
                                    ANTES del proyecto; no es de Nebula Arena) y su disco de 8 GB
Logs de CloudWatch ................ /aws/apigateway/welcome (de la cuenta, no del proyecto)

Etiqueta proyecto=nebula-arena: la API de etiquetas aún listaba 2 reglas de grupo de seguridad,
pero describe-security-group-rules responde "InvalidSecurityGroupRuleId.NotFound": ya no existen
(el índice de etiquetas tarda en actualizarse).

Se conservan a propósito (costo prácticamente cero):
- s3://nebula-tfstate-********1139  (estado de Terraform, para volver a crear todo)
- SSM /nebula/score-api-key          (llave de la tabla de líderes, SecureString estándar: gratis)
```

Figura 5. *Evidencia del terraform destroy y la revisión de recursos huérfanos en us-east-2. La instancia t3.micro que aparece es anterior al proyecto.*

Solo dejamos a propósito el bucket del estado y la llave en SSM, que cuestan prácticamente nada y permiten volver a crear todo con un comando.

Resultados: costo por hora

Consultamos los precios con la API de precios de AWS (us-east-2, 9 de noviembre):

Recurso	En reposo (2 nodos)	En el pico (6 nodos)
Plano de control de EKS	0.100	0.100
Nodos m7i-flex.large (USD 0.096 por hora c/u)	0.192	0.575
IPs públicas	≈ 0.020	≈ 0.040
Balanceador del matchmaker	0.025	0.025
Total (USD por hora)	**≈ 0.34**	**≈ 0.74**

En el pico de la 3.4 (252 jugadores) eso da **USD 0.003 por jugador y hora**. En reposo el clúster cuesta unos USD 8 al día, por eso lo destruimos al terminar cada sesión.

Conclusión

Terraform hizo que toda la plataforma se pudiera crear, revisar y borrar con comandos, sin pasos a mano. Revisar el plan antes de aplicar evitó sorpresas, y el plan sin drift y el segundo plan sin cambios demostraron que el código y la nube coincidían. Lo que más nos enseñó fue el `destroy`: borrar también se tiene que probar, porque encontró dos fallas reales que habrían dejado recursos cobrando.